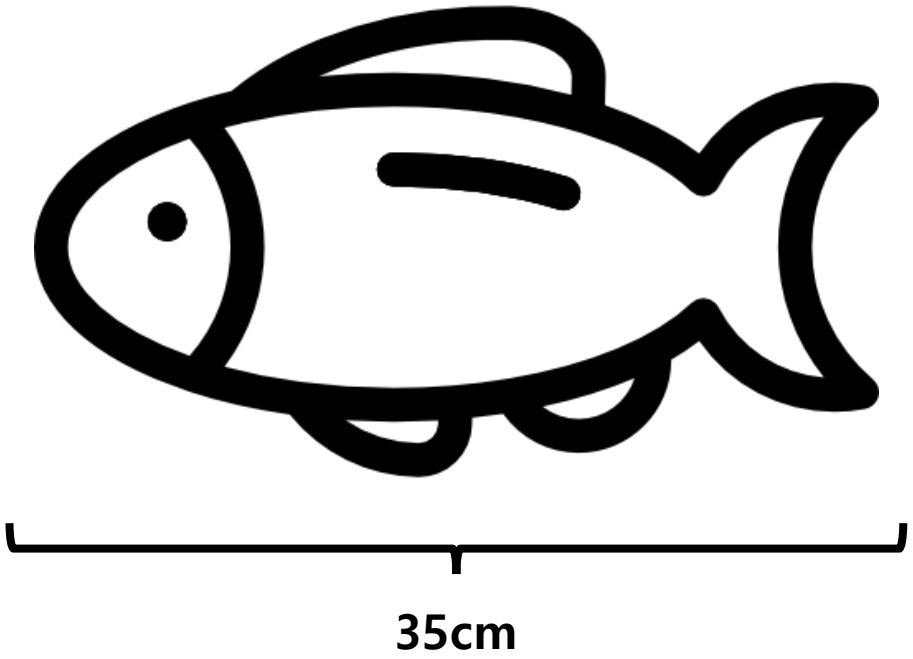
K 최근접 이웃 회귀와 모델 평가

농어의 무게를 예측할 수 있을까? (114p)



길이가 35cm인 농어의 무게를 예측 가능한가?

농어 56마리의 길이와 무게 데이터

K-최근접 이웃 회귀

분류와 회귀 (115p)

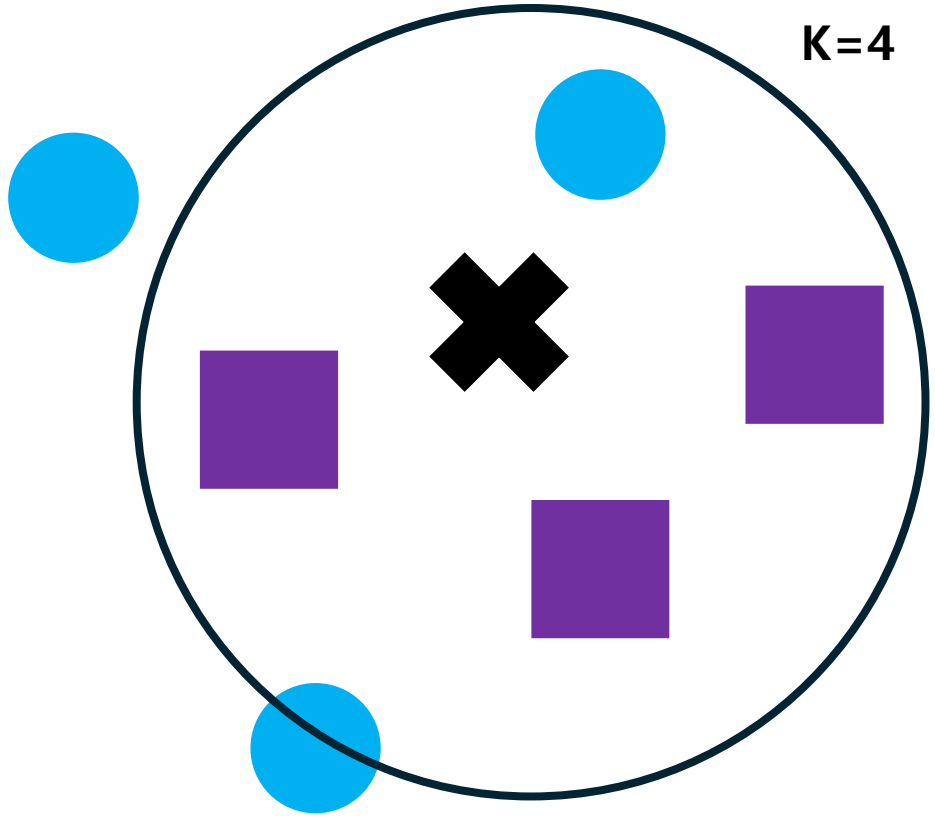
분류 (Classification) → 무엇인지 분류

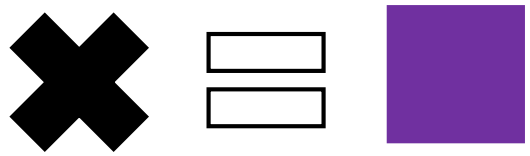
농어 / 빙어?

회귀 (Regression) → 얼마인지 예측

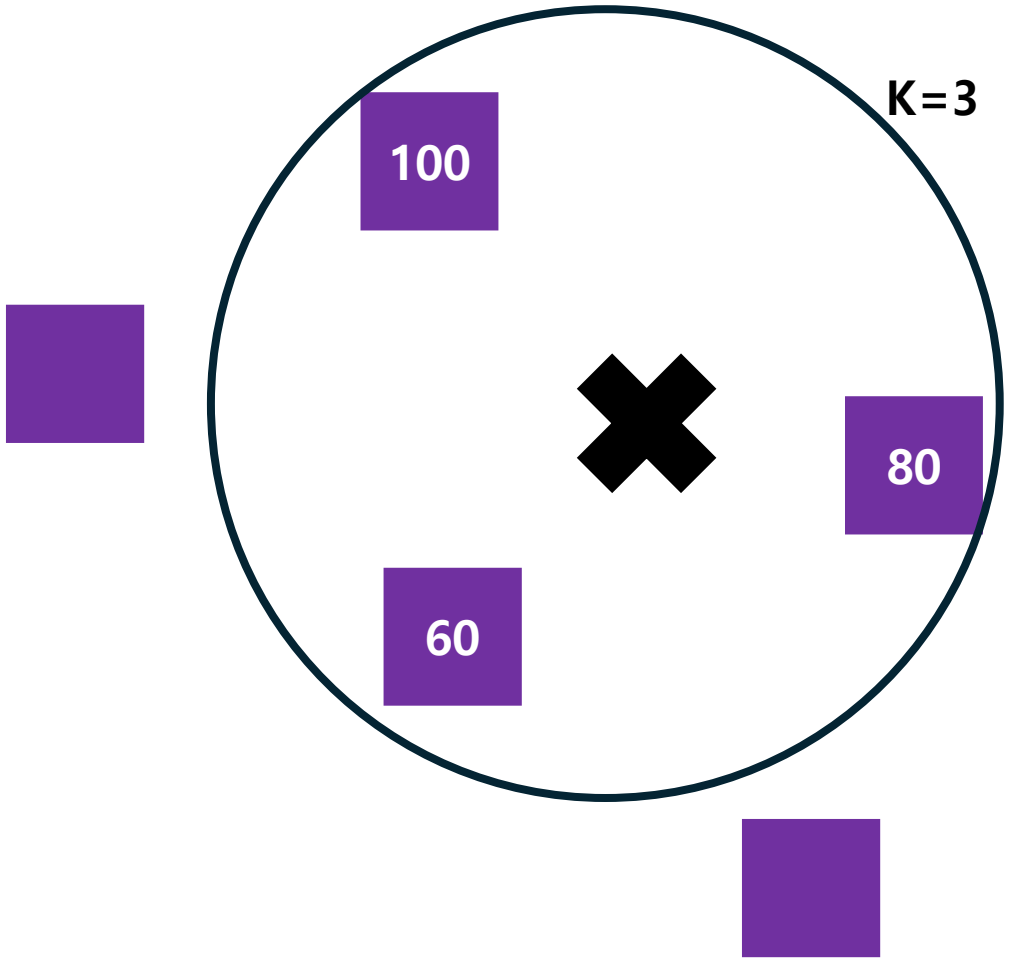
경제 성장률 예측, 배달 시간 예측

K-최근접 이웃 분류 알고리즘 (115p)





K-최근접 이웃 회귀 (116p)



✖ 의 최근접 이웃 샘플의 값은
60, 80, 100

✖ $\frac{60+80+100}{3} = 80$

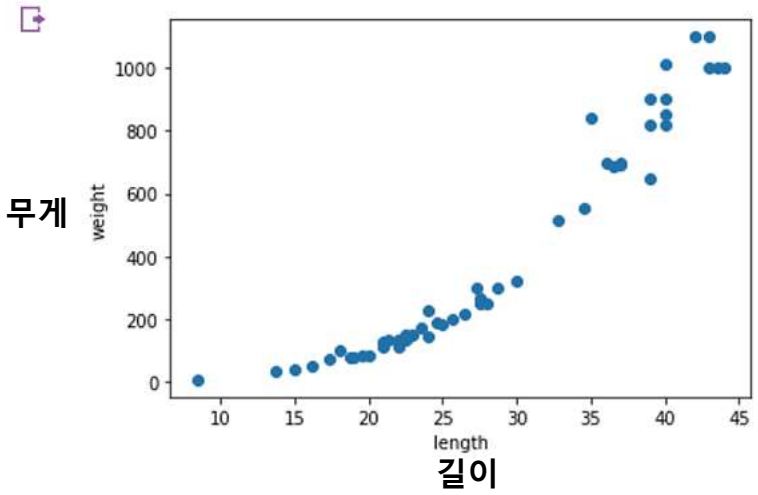
데이터 준비 (117p)

손코딩

```
import matplotlib.pyplot as plt
plt.scatter(perch_length, perch_weight)
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```

x = 농어의 길이

y = 농어의 무게



손코딩

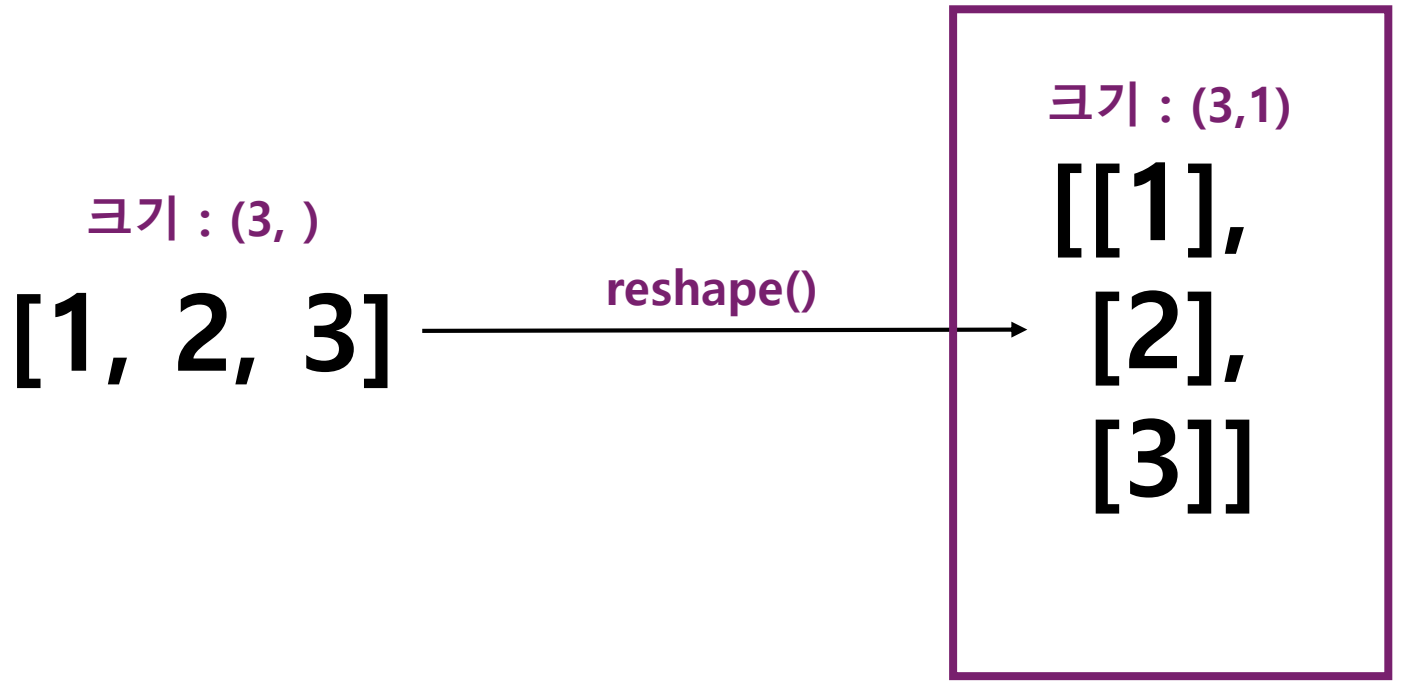
```
from sklearn.model_selection import train_test_split
train_input, test_input, train_target, test_target = train_test_split(
    perch_length, perch_weight, random_state=42)
```

훈련용 길이, 무게

테스트용 길이, 무게

기존 농어의 길이와 무게 샘플 데이터를 훈련용과 테스트용으로 나눈다.

reshape() (118p ~ 120p)



사이킷런에 사용 가능

arr.reshape(n, m) : arr을 n행 m열로 변환

reshape() (118 ~ 120p)

손코딩

```
test_array = np.array([1,2,3,4])
print(test_array.shape)
```

```
(4,)
```

[1, 2, 3, 4]

손코딩

```
test_array = test_array.reshape(2, 2)
print(test_array.shape)
```

```
(2, 2)
```

[[1, 2], [3, 4]]

reshape() (120p)

```
test_array = test_array.reshape(2, 3)
```

```
ValueError: cannot reshape array of size 4 into shape (2,3)
```

손코딩

```
train_input = train_input.reshape(-1, 1)
test_input = test_input.reshape(-1, 1)
print(train_input.shape, test_input.shape)
```

```
(42, 1) (14, 1)
```

NumPy가 해당 차원의 크기를 자동으로 계산

결정계수 R^2 (120p ~ 121p)

손코딩

```
from sklearn.neighbors import KNeighborsRegressor

knr = KNeighborsRegressor()

# k-최근접 이웃 회귀 모델을 훈련합니다
knr.fit(train_input, train_target)
```

K-최근접 이웃 분류
: KNeighborsClassifier

K-최근접 이웃 회귀
: KNeighborsRegressor

손코딩

```
print(knr.score(test_input, test_target))
```

```
0.9928094061010639
```

결정 계수 = R^2

결정계수 R^2 (120p ~ 121p)

$$R^2 = 1 - \frac{\sum(\text{타깃} - \boxed{\text{예측}})^2}{\sum(\text{타깃} - \text{평균})^2}$$

타깃의 평균 정도를 예측하는 수준 → 분자 ≈ 분**모** → $R^2 \approx 0$

타깃에 가깝게 예측하는 수준 → 분자 ≈ 0 → $R^2 \approx 1$

결정계수 R^2 (120p ~ 121p)

손코딩

```
from sklearn.metrics import mean_absolute_error

# 테스트 세트에 대한 예측을 만듭니다
test_prediction = knr.predict(test_input)

# 테스트 세트에 대한 평균 절댓값 오차를 계산합니다
mae = mean_absolute_error(test_target, test_prediction)
print(mae)
```

```
19.157142857142862
```

결과에서 예측이 평균적으로 19g 정도 타깃값과 다르다.

과대 적합 / 과소 적합 (122p)

훈련할 때 사용한 모델

손코딩

```
print(knr.score(train_input, train_target))
```

```
0.9698823289099255
```

보통 훈련 세트의 점수가 테스트 세트의 점수보다 높아야 한다.

과대 적합 / 과소 적합 (122p)

	훈련 세트	테스트 세트	원인
과대 적합 (Overfitting)	높은 점수	낮은 점수	훈련세트에만 맞는 모델
과소 적합 (Underfitting)	낮은 점수 (낮은 점수)	높은 점수 (낮은 점수)	모델이 너무 단순함

✚ 여기서 잠깐 왜 과소적합이 일어나죠?

이런 현상의 또 다른 원인은 훈련 세트와 테스트 세트의 크기가 매우 작기 때문입니다. 이 책에서는 독자들이 쉽게 이해할 수 있도록 지면에 모두 담을 수 있을 만큼의 작은 데이터를 사용했습니다. 데이터가 작으면 테스트 세트가 훈련 세트의 특징을 따르지 못할 수 있습니다. 하지만 여기서는 일반적인 경우를 설명합니다.

농어 무게 예측 모델에서 해결 (123p)

손코딩
```
print(knr.score(test_input, test_target))
```

```
0.9928094061010639
```

테스트 세트에서의 점수

손코딩
```
print(knr.score(train_input, train_target))
```

```
0.9698823289099255
```

훈련 세트에서의 점수

훈련 세트 < 테스트 세트

과소 적합

농어 무게 예측 모델에서 해결 (123p)

손코딩

```
# 이웃의 개수를 3으로 설정합니다
knr.n_neighbors = 3

# 모델을 다시 훈련합니다
knr.fit(train_input, train_target)
print(knr.score(train_input, train_target))
```

K값 낮게 설정 -> 훈련 세트의 국지적인 패턴에 민감해짐

```
0.9804899950518966
```

훈련 세트의 점수

손코딩

```
print(knr.score(test_input, test_target))
```

```
0.974645996398761
```

테스트 세트의 점수